

EXAM FLOW (Layered Architecture)

STEP 0 — Read the Question (1 minute)

Identify:

- **Entity Name**
 - **Movie**
 - **Fruit**
 - **Student**
 - **Employee**
 - **Book**
- **Fields**
 - **name**
 - **price**
 - **rating**
 - **city**
 - **etc.**
- **What is required?**
 - **Save only?**
 - **Save + Fetch All?**
 - **REST API?**
 - **JSP/Thymeleaf page?**
 - **Validation?**

Example

Entity : Movie

Fields:

movieName

rating

or

Entity : Fruit

Fields:

name

price

Everything else is almost the same.

STEP 1 — Create the Project

If Spring Boot:

- Spring Starter Project
- Maven
- Java

Dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Thymeleaf (if asked)
- Validation (if asked)

Run once.

If the application starts successfully,

 Continue.

STEP 2 — Create Packages (Layered Architecture)

Inside

src/main/java

Create:

controller

service

repository

entity

(Some colleges also create)

config

exception

dto

Only create them if required.

STEP 3 — Create Entity

Example:

Movie.java

Write:

- **@Entity**
- **@Id**
- **@GeneratedValue**
- **Fields**
- **Getters/Setters**
- **Constructors**

Example:

@Entity

public class Movie{

@Id

```
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int id;

private String movieName;

private int rating;

}
```

If validation is asked:

@NotBlank

@Min

@Max

STEP 4 — Create Repository

Create

MovieRepository

@Repository

```
public interface MovieRepository
extends JpaRepository<Movie,Integer>{

}
```

Done.

STEP 5 — Create Service Layer

Create

MovieService

Inject Repository

@Autowired

MovieRepository repository;

Create methods

save()

getAll()

If required

delete()

update()

STEP 6 — REST Controller (If API is Asked)

Create

MovieRestController

Write

@RestController

@RequestMapping("/movie")

Create API

@PostMapping("/save")

Return

service.save(movie);

If GET API is asked

@GetMapping("/getall")

Return

service.getAll();

STEP 7 — MVC Controller (If Page is Asked)

Create

MovieController

Write

@Controller

For display page

`@GetMapping("/getall")`

Add

`model.addAttribute("movies",
service.getAll());`

Return

`movies`

Meaning

`movies.html`

or

`movies.jsp`

depending on the question.

STEP 8 — Create View

If Thymeleaf

Create

templates

Inside

`movies.html`

Use

`th:each`

Display data

`th:text`

If JSP

Create

WEB-INF/views

Create

movies.jsp

Display using JSP/JSTL.

STEP 9 — Database Configuration

Write

spring.datasource.url=

spring.datasource.username=

spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

STEP 10 — Run Project

Run

No errors?

Good.

STEP 11 — Test POST API

Using Postman

POST

Example

localhost:8080/movie/save

Body

```
{  
  "movieName":"KGF",  
  "rating":9  
}
```

Click

Send

STEP 12 — Test GET

Browser

localhost:8080/getall

See table.

Finished.

★ Layered Architecture (Memorize This)

Client / Browser



Controller



Service



Repository (DAO)



Database

Data comes back in the reverse direction:

Database



Repository



Service



Controller



View (HTML/JSP)

DON'T PANIC IF THE ENTITY CHANGES

Tomorrow they may ask:

- **Fruit**
- **Movie**
- **Student**
- **Employee**
- **Product**
- **Book**

Only change:

Movie



Fruit

and

movieName



fruitName

The project structure and flow remain the same.